

Trabajo Integrador 2026

Seminario de Lenguajes Opción Python

El trabajo integrador consta del desarrollo de una aplicación de análisis y visualización de información relacionada al estándar Darwin Core, a partir de este momento nos referiremos como a la aplicación como la app. El desarrollo de este trabajo involucra el análisis de los datos con los cuales se trabajarán, su limpieza y preparación necesaria y finaliza con el desarrollo de una interfaz amigable con el usuario/a. El objetivo es que logren aplicar sus conocimientos de programación mientras exploran datos reales y construyen una interfaz visual con Streamlit.

El proyecto se divide en dos etapas:

- **Primera Etapa:** se enfocarán en entender y sanitizar el conjunto de datos provenientes de <https://biodiversidad.ar/> . Tendrán que trabajar con conjuntos de datos relacionados al avistamiento de especies del reino animal y/o vegetal
- **Segunda Etapa:** diseñarán la interfaz en Streamlit e integrarán los datos procesados. Se crearán diferentes secciones para mostrar información relevante sobre los datos del sistema.

Este trabajo será realizado en grupos de **cinco** personas, por lo que se espera una distribución equitativa de tareas. Cada integrante del equipo tendrá un rol importante en el desarrollo, ya sea en la manipulación de datos, programación de funcionalidades, diseño de la interfaz o documentación del proyecto.

Además de mejorar su manejo de Python y librerías como *Pandas*, *Matplotlib* y *Streamlit* (entre otras), este proyecto les permitirá desarrollar habilidades de trabajo en equipo, control de versiones con *Git* (*GitLab*) y documentación de código.

El presente documento describe el trabajo a realizar y los requerimientos de la segunda parte.

Antes de comenzar

Para la realización de la parte 2 del trabajo integrador se asume comprendida la parte 1 del mismo, debido que comenzaremos el análisis y visualización de información se recomienda en caso de dudas repasar lo realizado para la parte 1.

En la mencionada parte 1 del trabajo integrador se solicitó crear una aplicación en Streamlit que contenga un menú lateral (**sidebar**) con las siguientes páginas:

- (P1) Inicio
- (P2) Estado del sistema
- (P3) Búsqueda (sin implementar)
- (P4) Visualización (sin implementar)

En la parte 2 le daremos funcionalidad a cada una de estas páginas.

Carga y selección de datasets

Consideraciones iniciales

- Los datasets a cargar son los generados en `processed_datasets` durante la primera etapa.
- Se permite y recomienda utilizar `st.session_state` para almacenar información útil entre páginas.

Ejercicio 1.A

Implementar una función que reciba el nombre de un dataset y lo cargue como un DataFrame de pandas.

Para esta segunda etapa permitiremos que se asuma que los datasets tienen un tamaño lo suficientemente pequeño como para cargarlo en memoria, por lo tanto NO es necesario que utilicen chunks en Pandas. Pueden crear el dataframe en pandas del archivo completo sin problemas.

La función debe:

- Recibir el nombre del dataset.
- Respetar el separador y encoding identificados en la primera etapa.
- Retornar el DataFrame resultante.

Ejercicio 1.B

En el sidebar de la aplicación, agregar un selector que permita al usuario elegir con cuál de los datasets disponibles desea trabajar. La selección deberá persistir entre páginas durante la sesión.

Ejercicio 1.C

Agregar en el sidebar un indicador que muestre:

- Nombre del dataset actualmente seleccionado.
- Cantidad de registros del dataset.
- Fecha y hora de la selección.

Página de Búsqueda (P3)

Objetivo de la sección

En esta sección se completará la página de Búsqueda (P3), que fue definida como placeholder en la primera etapa. El objetivo es permitir al usuario localizar registros específicos dentro del dataset a partir de distintos criterios combinables.

Consideraciones iniciales

- Reutilizar las funciones de búsqueda desarrolladas en el Ejercicio 5.A de la primera etapa.
- El resultado debe mostrarse como una tabla interactiva.

Ejercicio 2.A

Implementar un campo de búsqueda por texto libre que busque coincidencias en un campo seleccionado por el usuario, es decir, el usuario selecciona la columna por la que desea buscar y luego el valor que pretende encontrar.

La búsqueda debe ser case-insensitive y debe retornar todos los registros que contengan el texto ingresado como subcadena.

Ejercicio 2.B

Añadir los siguientes filtros a la página de búsqueda:

- Nombre científico: selector múltiple con los valores disponibles en el dataset.
- Observador: Texto libre.
- País: selector múltiple con los valores disponibles en el dataset.
- Provincia: selector múltiple con los valores disponibles en el dataset.
- Rango de búsqueda: seleccionar dos fechas y filtrar solo ocurrencias dentro del intervalo. Si estos datos están disponibles en el dataset, se debe bloquear esta búsqueda.

Consideraciones:

- Los filtros deben poder combinarse: el resultado es la intersección de todas las condiciones activas.
- Se deben mapear los nombres a los atributos reales del dataframe.
- En caso de tener valores vacíos esa "fila" se descarta de la búsqueda.
- No se debe tener las opciones para los selectores definida de manera *hardcodeada* en el código, debe generarse a partir de la lectura y análisis de cada dataset.

Ejercicio 2.C

Mostrar los resultados en un componente que debe:

- Permitir ordenamiento por columna.
- Mostrar únicamente columnas relevantes (a criterio del grupo, justificando la elección en la documentación).
- Indicar la cantidad de registros encontrados.
- Pagar el resultado.
- Acceder al detalle de un registro.

Ejercicio 2.D

Agregar un botón "Exportar resultados" que permita descargar como CSV el subconjunto de registros actualmente visible.

El nombre del archivo deberá contener el nombre del dataset y la fecha de descarga.

Ejercicio 2.E

Debajo de la tabla (no se debe exportar) añadir un resumen estadístico del subconjunto filtrado que muestre:

- Cantidad de especies únicas (scientificName).
- Países representados.
- Provincias representadas.
- Cantidad de observadores que aportaron sus hallazgos.

Página de Visualización (P4)

Objetivo de la sección

En esta sección se implementará la página de Visualización (P4). El objetivo es presentar de forma gráfica la distribución y características de los datos de biodiversidad mediante gráficos estadísticos y una visualización geográfica.

Consideraciones iniciales

- Se pueden utilizar las librerías matplotlib, plotly o altair para la generación de gráficos.
- Cada gráfico debe incluir título, etiquetas de ejes y leyenda cuando corresponda.
- Los gráficos deben responder al dataset seleccionado en el sidebar.

Ejercicio 3.A

Implementar un gráfico de barras que muestre la cantidad de registros por

- country: en los datasets que tengan registros de más de un country.
- stateProvince/locality: en los datasets que tengas registros de un mismo país

Agregar un control deslizante (st.slider) que permita al usuario elegir cuántos países mostrar (los N más frecuentes).

Ejercicio 3.B

Implementar un gráfico de líneas que muestre la cantidad de registros por año, basándose en eventDate.

Manejar adecuadamente los registros con fechas nulas o en formato inconsistente, informando al usuario cuántos registros fueron excluidos.

Ejercicio 3.C

Implementar un gráfico que muestre la distribución de registros por nivel taxonómico. Agregar un selector que permita al usuario elegir entre:

- Clase (class)
- Orden (order)
- Familia (family)

Ejercicio 3.D

Implementar una visualización que muestre el porcentaje de registros no nulos por columna, representado como un gráfico de barras horizontales ordenado de mayor a menor completitud.

Reutilizar la función desarrollada en el Ejercicio 2.F de la primera etapa.

Ejercicio 3.E

Cuando haya más de un dataset disponible en processed_datasets, mostrar una tabla comparativa que incluya:

- Cantidad de registros por dataset.
- Porcentaje de registros con coordenadas válidas.
- Porcentaje de registros con fecha válida.

- Porcentaje de campos completados en promedio.

Gestión de registros (P5)

Objetivo de la sección

En esta sección se implementará una nueva página (P5 – Gestión de registros) que permita realizar operaciones de inserción, actualización y eliminación de registros directamente desde la interfaz.

Todas las operaciones deben reutilizar las funciones de los Ejercicios 4, 5 y 6 de la primera etapa. Cada operación debe generar el archivo correspondiente en `processed_datasets` y registrar la entrada en el log de operaciones.

Consideraciones iniciales

- Validar los datos ingresados antes de ejecutar cualquier operación.
- Informar al usuario sobre el resultado de cada operación (exitoso,)
- Las operaciones deben aplicarse siempre sobre el dataset actualmente seleccionado.

Ejercicio 4.A

Implementar un formulario para ingresar los datos de un nuevo registro.

El formulario debe:

- Contener campos para los atributos principales del dataset.
- Validar los datos al enviar, reutilizando las funciones del Ejercicio 4.C de la primera etapa.
- Mostrar mensajes de error específicos por campo en caso de validación fallida.
- Al confirmar, invocar las funciones de inserción de la primera etapa y reflejar el cambio en el dataset activo.

Ejercicio 4.B

Antes de modificar un registro el usuario debe localizarlo. Implementar una búsqueda por identificador (el identificador depende del dataset) que muestre el registro encontrado en un formulario editable.

Si el identificador no existe, informar al usuario de forma clara.

Ejercicio 4.C

A partir del registro localizado en el ejercicio anterior, mostrar un formulario que permita editar sus campos.

El formulario debe:

- Presentar los valores actuales como valores por defecto de cada campo.
- Validar los nuevos valores antes de aplicar el cambio, reutilizando las funciones del Ejercicio 5.D de la primera etapa.
- Invocar las funciones de actualización de la primera etapa al confirmar.
- Mostrar un resumen de los campos modificados.

Ejercicio 4.D

Implementar una sección con tres modalidades de eliminación:

- Por identificador único (equivalente al Ejercicio 6.A de la primera etapa).
- Por valor de una columna, aceptando uno o varios valores (equivalente al Ejercicio 6.B de la primera etapa).
- Por condición sobre una columna, aceptando operadores `==`, `!=`, `>`, `>=`, `<`, `<=` (equivalente al Ejercicio 6.C de la primera etapa).

Antes de ejecutar la eliminación, mostrar al usuario una vista previa de los registros que serán afectados y solicitar confirmación explícita.

Ejercicio 4.E

Extender la página P2 – Estado del sistema para que, además de mostrar el log, permita:

- Filtrar por tipo de operación (INSERT, UPDATE, DELETE, ERROR).

- Filtrar por rango de fechas.
- Mostrar métricas resumidas: total de inserciones, actualizaciones, eliminaciones y errores registrados.

Exploración de datasets (P6)

Objetivo de la sección

En esta sección se implementará la página (P6 – Datasets), cuyo propósito es ofrecer una vista general de todos los datasets disponibles en el sistema. A diferencia del resto de las páginas, esta no depende del dataset seleccionado en el sidebar.

Ejercicio 5.A

Mostrar una tabla con todos los datasets disponibles. Para cada uno incluir:

- Nombre del archivo.
- Tamaño en disco (en MB).
- Fecha de última modificación.

Ejercicio 5.B

Agregar un selector que permita al usuario elegir uno de los datasets listados para ver su información detallada. Al seleccionarlo, mostrar:

- Lista completa de columnas.
- Cantidad total de registros.
- Porcentaje de valores nulos por columna.

Ejercicio 5.C

Mostrar una tabla comparativa de todos los datasets. La tabla debe incluir:

- Cantidad de registros por dataset.
- Porcentaje de registros con coordenadas válidas.

- Porcentaje de registros con fecha válida.
- Porcentaje de campos completados en promedio.

Ejercicio 5.D

Si el grupo generó archivos de documentación en formato Markdown durante la primera etapa (carpeta documentation), mostrar su contenido en esta página utilizando `st.markdown`.

Agregar un selector para que el usuario elija cuál de los documentos disponibles visualizar.

Página de Ficha de datos (P7)

Objetivo de la sección

En esta sección se implementará la página (P7 – Ficha de datos) que no se mostrará en el sidebar y, cuyo propósito es ofrecer una mapa con las observaciones/ocurrencias seleccionadas en la página de búsqueda. Al seleccionar una ocurrencia se deberá mostrar una ficha de datos de la observación ocurrencia seleccionada y la información disponible (imagen o sonido) para esa ocurrencia (si es que la hay). Esta página usará los criterios de búsqueda de P3.

Consideraciones iniciales

- Para la visualización geográfica se recomienda folium.

Ejercicio 6.A

Crear la página y configurarla para que no se muestre en el sidebar

Agregue en la página de búsqueda P3 un botón “Ver ficha” que redirija a esta página pasando de alguna manera la información que sea necesaria de las ocurrencias seleccionadas (por ejemplo a través de la sesión o de query parameters)..

Ayuda:

- <https://discuss.streamlit.io/t/how-to-redirect-to-another-page-after-login-or-dont-show-the-list-of-pages-on-the-sidebar/61863/4>

- https://docs.streamlit.io/develop/api-reference/navigation/st.switch_page

Ejercicio 6.B

Implementar un mapa interactivo que muestre la ubicación geográfica de los registros resultantes de la búsqueda de P3 usando `decimalLatitude` y `decimalLongitude`.

El mapa debe:

- Excluir registros con coordenadas inválidas o nulas (reutilizar las validaciones del Ejercicio 3 de la primera etapa).
- Permitir colorear los puntos según un criterio elegido por el usuario (por ejemplo: por país, por especie o por institución).
- Informar cuántos registros fueron excluidos del mapa por coordenadas inválidas.

Ejercicio 6.C

Al seleccionar un punto en el mapa se deberá mostrar una tabla (estilo ficha de wikipedia) con la información taxonómica de la especie observada, lugar y fecha de la observación.

Ejercicio 6.D

Al seleccionar un punto en el mapa se deberá mostrar toda la información multimedia relacionada a la ocurrencia correspondiente. En el caso del dataset Xeno-Canto mostrar solamente los archivos de sonido usando un reproductor multimedia por sonido. En el caso de iNaturalist mostrar cada foto.

Para esto será necesario:

- Cruzar datos core con multimedia por id
- Detectar el tipo de multimedia
- Filtrar espectrogramas y similares de Xeno-Canto para quedarnos sólo con sonidos.

Consideraciones de la entrega y defensa

Si bien el trabajo es grupal, **la nota de la defensa es INDIVIDUAL.**

La defensa se lleva a cabo durante el horario de consulta la semana posterior a la fecha de entrega. Este proceso implica la realización de un encuentro virtual o presencial con el ayudante asignado, durante el cual se formulan una serie de preguntas relacionadas con la entrega. Dichas preguntas están dirigidas a los distintos miembros del grupo, distribuyendo

equitativamente la participación entre todos ellos. La finalidad es que cada integrante tenga la oportunidad de responder. La evaluación se basa en el desempeño de las respuestas, lo que determina finalmente la nota asignada a cada miembro del grupo en relación con la entrega realizada.

Esto implica que, aunque se trate de una entrega grupal, las calificaciones entre los miembros del grupo pueden variar.

Tener en cuenta que es fundamental mantener una participación activa tanto en las consultas prácticas como en el repositorio de GitLab. Esto permitirá al ayudante obtener una comprensión conceptual del conocimiento de cada participante del grupo.

Detalles de entrega

La fecha de entrega de la primera etapa es el día viernes **5 de junio a las 23:59 hs.** **La defensa será la semana del 8 de junio en horario de la consulta práctica.**

La segunda entrega otorga al estudiante un **máximo de 40 puntos**, los cuales se basan en su desempeño individual durante las consultas y la defensa.

Criterios de Evaluación

La distribución de los puntos de la entrega estará disponible en la rúbrica asociada a la tarea. Se evaluará:

- Funcionalidad implementada de acuerdo al enunciado.
- Cumplimiento de las consideraciones planteadas.
- Código subido en tiempo y forma al repositorio de GitLab indicado.
- Participación individual activa durante el desarrollo del trabajo, que incluye asistencia a las consultas (tanto virtuales como presenciales) y contribuciones al repositorio de GitLab.
- En la defensa, se espera que cada integrante del grupo demuestre los conocimientos utilizados para la realización del trabajo.